

Lecture 22: Touch the Network

Scapy, Mininet, and Sockets

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Session 1 — Craft and Emulate

- Scapy: build packets in Python
- ICMP, TCP SYN, traceroute
- Mininet: real topologies in a VM
- iperf across a bottleneck

Session 2 — Applications

- The socket API
- TCP echo server + client
- UDP echo server + client
- setsockopt, Nagle
- Close the loop: sockets → pcap → analysis

Theme

Last week you watched packets fly. Today you launch them.

Three tools, one narrative:

- ① **Scapy** — craft a single packet, byte by byte.
- ② **Mininet** — own a whole topology.
- ③ **Sockets** — the interface real applications use.

At each stage you are an active participant, not an observer. By the end you can both *generate* traffic and *inspect* it — the full read + write loop.

Everything runs inside the ec441 Multipass VM. Scapy and Mininet use raw sockets and network namespaces — Linux only, and you need `sudo`.

What is Scapy?

A Python library for building, sending, receiving, and dissecting packets at *any* layer.

Think “Wireshark in reverse”:

- Wireshark: bytes on the wire → Python objects
- Scapy: Python objects → bytes on the wire

Why use it? Fastest way to test a network hypothesis. *“What does this router do with a packet I built by hand?”* Build one and find out.

Setup

```
sudo pip install scapy  
sudo python3      # raw sockets require root
```

Scapy uses `AF_PACKET` / `SOCK_RAW` to bypass the kernel's normal socket layer — so every send and every sniff requires root.

```
from scapy.all import IP, ICMP

pkt = IP(dst="8.8.8.8") / ICMP()
pkt.show()           # pretty-print every field
bytes(pkt)           # the actual wire bytes
```

- The / operator is Scapy's layer-stacking syntax: read it as “*encapsulated in.*”
- IP() / ICMP() = ICMP inside IP.
- Scapy fills in defaults: version=4, ihl=5, ttl=64, proto=1 (ICMP).

See: demo_scapy_build_l22.py

Scapy — Ping from Scratch

```
from scapy.all import IP, ICMP, sr1

req = IP(dst="8.8.8.8") / ICMP()
reply = sr1(req, timeout=2, verbose=0)
reply.show()
```

- sr1 = “send and receive one.”
- Reply TTL tells you how many hops back — count down from 64 / 128.
- Compare against `ping -c 1 8.8.8.8`: same ICMP, nicer CLI.

See: `demo_scapy_icmp_l22.py`

```
from scapy.all import IP, TCP, sr1, RandShort

r = sr1(IP(dst="scanme.nmap.org")
        / TCP(dport=80, flags="S", sport=RandShort()),
        timeout=2, verbose=0)
print(r.sprintf("%TCP.flags%"))    # 'SA' = SYN-ACK = open
```

- Port 80 → SA: server is listening.
- Port 81 → R / RA: no listener (L18 review).
- This is the core of a port scanner in four lines.

Ethics: Scanning hosts you do not own may violate US CFAA. `scanme.nmap.org` and your own VMs are fine; everything else needs written permission.

Scapy — Traceroute in 10 Lines

```
from scapy.all import IP, ICMP, sr1

for ttl in range(1, 16):
    r = sr1(IP(dst="8.8.8.8", ttl=ttl) / ICMP(),
            timeout=2, verbose=0)
    if r is None:
        print(f"{ttl:2d}__")
    elif r.type == 0:                                # Echo Reply = reached
        print(f"{ttl:2d}__{r.src}__(reached)"); break
    else:                                             # Time Exceeded
        print(f"{ttl:2d}__{r.src}")
```

TTL exhaustion is not a feature — it is a side effect of IP forwarding (L13, L17). Every path-discovery tool on earth runs this loop.

See: `demo_scapy_traceroute_l22.py`

```
from scapy.all import sniff

pkts = sniff(filter="icmp", count=5, iface="any")
pkts.summary()
```

- Same libpcap / BPF filter syntax as tcpdump and pyshark.
- **Scapy**: send AND capture in one script.
- **pyshark** (L21): bulk read-only analysis.
- **tcpdump** (L21): quick CLI capture.

What is Mininet?

Mininet creates virtual hosts using **Linux network namespaces** and connects them with virtual Ethernet pairs (`veth`).

- Each “host” is a real Linux process with its own network stack.
- It is not a simulation — it is a real network, in miniature, on one kernel.
- Ten lines of Python → a topology with controllable bandwidth, delay, and loss.

This is how researchers (and engineers at Google, Cisco, etc.) test protocol changes before shipping them to physical routers.

```
sudo mn --topo single,3 --link tc  
  
mininet> h1 ping -c 3 h2  
mininet> pingall  
mininet> exit
```

Three hosts attached to one switch. `--link tc` uses Linux traffic-control for link emulation. `pingall` verifies full reachability.

Mininet — Bottleneck Topology

```
from mininet.net import Mininet
from mininet.link import TCLink
from mininet.node import OVSController

net = Mininet(controller=OVSController, link=TCLink)
net.addController('c0')
h1 = net.addHost('h1'); h2 = net.addHost('h2')
s1 = net.addSwitch('s1')

net.addLink(h1, s1, bw=100, delay='1ms')
net.addLink(s1, h2, bw=10, delay='20ms', loss=1)    # bottleneck

net.start()
h2.cmd('iperf3_-s_-D')
print(h1.cmd('iperf3_-c_-%s_-t_20' % h2.IP()))
net.stop()
```

See: `demo_mininet_bottleneck_l22.py`

Running iperf across the bottleneck:

- **Bandwidth:** settles near 10 Mb/s — bottleneck governs.
- **Loss:** 1% triggers fast retransmit and multiplicative decrease (L20); throughput noticeably below the loss-free ceiling.
- **Delay:** 20 ms one-way $\Rightarrow$ 40 ms RTT.
- **BDP:** $10 \text{ Mb/s} \times 40 \text{ ms} \approx 50 \text{ KB} \approx 35 \text{ MSS}$.

Contrast with L21 loopback iperf

Loopback had sub-millisecond RTT and no loss — cwnd stayed at $iw=10$ forever. **Mininet is where the L19/L20 material actually shows up in ss -tipm.**

The Socket API

Every networked program you have ever used sits on top of this API: browsers, SSH, dig, curl, Zoom, games.

TCP server flow:

`socket() → bind() → listen() → accept() → recv()/send() → close()`

TCP client flow:

`socket() → connect() → send()/recv() → close()`

UDP server flow:

`socket() → bind() → recvfrom()/sendto() → close()`

UDP client flow:

`socket() → sendto()/recvfrom() → close()`

TCP vs UDP API at a Glance

	TCP	UDP
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection setup	listen/accept/connect	none
Message boundaries	byte stream (framed by app)	preserved per datagram
Retransmission	kernel	application's job
Ordering	guaranteed	none
Typical users	HTTP, SSH, Git	DNS, QUIC, video, games


```
import socket
HOST, PORT = '127.0.0.1', 5001

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind((HOST, PORT))
srv.listen(1)

while True:
    conn, addr = srv.accept()
    with conn:
        while True:
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data)
```

See: `demo_tcp_echo_server_122.py`

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(('127.0.0.1', 5001))
    s.sendall(b'hello_from_ec441\n')
    print(s.recv(1024))
```

- `SO_REUSEADDR`: without it, restart within ~ 60 s fails with *“Address already in use”* (`TIME_WAIT` — L19).
- `recv(1024)` is a *hint*. TCP is a byte stream — you may get 1 to 1024 bytes per call.
- `sendall` vs `send`: `sendall` loops until everything is written.

UDP Echo — Server and Client

Server

```
import socket
srv = socket.socket(socket.AF_INET,
                    socket.SOCK_DGRAM)
srv.bind(('127.0.0.1', 5002))
while True:
    data, addr = srv.recvfrom(1024)
    srv.sendto(data, addr)
```

Client

```
import socket
s = socket.socket(socket.AF_INET,
                  socket.SOCK_DGRAM)
s.sendto(b'hello_udp\n',
        ('127.0.0.1', 5002))
print(s.recvfrom(1024))
```

Gone: listen, accept, connect, per-connection socket. UDP is stateless at the socket layer.

See: `demo_udp_echo_{server,client}_l22.py`

setsockopt — Options Worth Knowing

Option	Effect	Set when
SO_REUSEADDR	Immediate rebind after close	always on servers
TCP_NODELAY	Disable Nagle's algorithm	interactive / RPC
SO_KEEPALIVE	Periodic probes on idle TCP	long-lived connections
SO_RCVBUF / SNDBUF	Socket buffer size	high-BDP paths

Nagle's Algorithm

Nagle batches small writes into bigger TCP segments to cut per-packet overhead. Great for bulk transfer, terrible for interactive sessions (ssh, RPC).

Default is *on*; disable per socket with TCP_NODELAY.

Close the Loop

Run everything together in the VM:

```
# shell 1 -- capture
sudo tcpdump -i lo -w /tmp/sockets.pcap \
    'port 5001 or port 5002'

# shell 2 -- servers
python3 demo_tcp_echo_server_122.py &
python3 demo_udp_echo_server_122.py &

# shell 3 -- clients
python3 demo_tcp_echo_client_122.py
python3 demo_udp_echo_client_122.py

# shell 1 -- stop
sudo pkill tcpdump
```

Open `/tmp/sockets.pcap` in pyshark or Wireshark — you wrote the app, and now you are analyzing its traffic.

Every Layer, One Run

The `sockets.pcap` you just captured contains:

- **Ethernet frame** (L08) carrying
- **IP datagram** (L14–L17) carrying
- **TCP segment with cwnd, seq/ack, RTT** (L18–L20) carrying
- **application bytes** from *your* Python code.

Today's payoff

You wrote the application. You emulated the network path. You captured the traffic. You can explain every field.

What's Next

Lecture	Date	Topic
L23	Thu Apr 23	Application layer — HTTP, DNS, QUIC
L24	Tue Apr 28	Cryptography and security
L25	Thu Apr 30	Project demo day

L23 Preview

HTTP is a protocol you could now write from sockets. L23: why you wouldn't want to — and how DNS, HTTP/1.1, HTTP/2, HTTP/3 / QUIC actually work.